

NEXUS FILE EXPLORER

Complete Development Manual

Everything you need to build, test, and maintain the project using Claude Code.

From first install to final deployment.

Table of Contents

1. Prerequisites and Downloads
2. Computer Setup
3. Understanding Claude Code
4. Project Scaffolding
5. The Safe Development Environment
6. The Build Pipeline (Phase by Phase)
7. How to Prompt Claude Code
8. Testing Your Application
9. Making Edits Efficiently
10. Version Control with Git
11. Sharing the Project
12. Updating and Maintaining the App
13. Deploying to Other Machines
14. Troubleshooting Common Issues
15. Reference: File Structure and Tech Stack
16. Reference: Keyboard Shortcuts and Commands

1. Prerequisites and Downloads

1.1 Required Software

Before writing a single line of code, install all of the following on your Windows machine. Each tool serves a specific role in the development pipeline.

Python 3.11+

The entire backend of the application runs on Python. You need version 3.11 or newer.

Step 1

Go to python.org/downloads and download the latest Python 3.12+ installer for Windows.

Step 2

Run the installer. **Check the box that says "Add Python to PATH"** on the first screen. This is critical.

Step 3

Click Install Now (default settings are fine).

Step 4

Open a new terminal (PowerShell or Command Prompt) and verify:

```
python --version
pip --version
```

Step 5

Both should return version numbers. If python is not recognized, you missed the PATH checkbox. Rerun the installer and check it.

WARNING

Do not use the Microsoft Store version of Python. It creates PATH and permission issues. Use the official python.org installer.

Node.js 20+

Required for Claude Code and for the Electron frontend.

Step 1

Go to nodejs.org and download the LTS version (20+).

Step 2

Run the installer with default settings.

Step 3

Verify in a new terminal:

```
node --version  
npm --version
```

Git

Version control. Every code change gets tracked. Non-negotiable for a project this size.

Step 1

Download from **git-scm.com**.

Step 2

Install with default settings. When it asks about the default editor, pick whatever you are comfortable with (VS Code recommended, or Nano if unsure).

Step 3

Verify:

```
git --version
```

Visual Studio Code

Your code editor. Claude Code writes the code, but you need VS Code to browse files, read errors, and make quick manual edits.

Step 1

Download from **code.visualstudio.com**.

Step 2

Install. Check "Add to PATH" during setup.

Step 3

Install these extensions from the Extensions panel (Ctrl+Shift+X):

- **Python** (Microsoft)
- **Pylance** (Microsoft)
- **SQLite Viewer** (for inspecting your database)
- **GitLens** (for visual Git history)
- **Prettier** (for frontend code formatting)

Claude Code

The AI coding agent from Anthropic that will write most of your code. It runs in your terminal and has full access to your project files.

Step 1

Open a terminal and install globally via npm:

```
npm install -g @anthropic-ai/claude-code
```

Step 2

Authenticate with your Anthropic account:

```
claude login
```

Step 3

This opens a browser window. Log in with your Anthropic account. You need an active API plan or a Max subscription.

Step 4

Verify it works:

```
claude "say hello"
```

TIP

Claude Code bills against your Anthropic API usage. If you have a Claude Pro/Max subscription, it may be included. Check your plan details at console.anthropic.com.

Ollama (Local LLM Runtime)

Runs AI models locally on your machine. This is what powers the file classification engine without sending data to the cloud.

Step 1

Download from ollama.com. Run the installer.

Step 2

Open a terminal and pull the model you will use:

```
ollama pull llama3.1:8b
```

Step 3

Test that it runs:

```
ollama run llama3.1:8b "Classify this file: budget_2024_final.xlsx"
```

Step 4

It should respond with a reasonable classification. Close it with `/bye`.

With your hardware, the 8B parameter model will run comfortably. If you want better accuracy and have 16GB+ VRAM, try `llama3.1:70b` (much slower, much smarter).

1.2 Paid Services

Anthropic API key: required for Claude Code and for the cloud classification fallback. Get one at console.anthropic.com. Add a payment method and set a usage limit you are comfortable with. For development, \$20/month is more than enough. The API key is a string starting with 'sk-ant-'. Store it securely. Never commit it to Git.

TIP

Create a file called `.env` in your project root with `ANTHROPIC_API_KEY=sk-ant-your-key-here`. The app reads from this file. Git will ignore it (we set that up later).

1.3 Optional But Recommended

- **Windows Terminal:** better than default Command Prompt. Install from Microsoft Store. Supports tabs, split panes, better fonts.
- **Hyper-V or VirtualBox:** for the safe testing VM. Hyper-V is built into Windows Pro (enable via Windows Features). VirtualBox is free and works on Windows Home.
- **DB Browser for SQLite:** GUI tool for inspecting/editing SQLite databases. Download from sqlitebrowser.org.

2. Computer Setup

2.1 Create Your Development Workspace

Pick a permanent home for your project. Everything related to Nexus lives here.

```
mkdir C:\Dev
mkdir C:\Dev\nexus
cd C:\Dev\nexus
```

This is your project root. All Claude Code sessions will start from this directory.

2.2 Create the Test Environment

You need a fake filesystem to test against so you never risk your real files.

Step 1

Create the test sandbox:

```
mkdir C:\NexusTestSandbox
mkdir C:\NexusTestSandbox\Downloads
mkdir C:\NexusTestSandbox\Documents
mkdir C:\NexusTestSandbox\Pictures
mkdir C:\NexusTestSandbox\Music
mkdir C:\NexusTestSandbox\Videos
mkdir C:\NexusTestSandbox\Projects
mkdir C:\NexusTestSandbox\Random
mkdir C:\NexusTestSandbox\AppData
```

Step 2

Populate it with dummy files. Create a script or manually drop in a mix of file types: images, PDFs, text files, code files, audio files, duplicates, files with vague names, empty folders, deeply nested folders.

Step 3

Create a snapshot of this sandbox before testing. Copy the entire NexusTestSandbox folder to NexusTestSandbox-backup. If a test goes wrong, delete the sandbox and restore from backup.

WARNING

Every path in the Nexus codebase must be a configuration variable, not hardcoded. During development, all paths point to C:\NexusTestSandbox. You only switch to real paths after thorough testing.

2.3 Set Up the Virtual Machine (For Destructive Testing)

For testing features that delete files, modify the registry, or register as default folder handler, use a VM.

Step 1

If you have Windows Pro: enable Hyper-V in Windows Features, create a new VM, install Windows on it.

Step 2

If you have Windows Home: install VirtualBox from [virtualbox.org](https://www.virtualbox.org), create a VM, install Windows using a free evaluation ISO from Microsoft.

Step 3

Once the VM has Windows running, install Python, Node.js, and Git inside the VM (same versions as your host).

Step 4

Take a VM snapshot called 'clean-state'. Before every round of destructive testing, revert to this snapshot.

Step 5

Share your project folder between host and VM using a shared folder (VirtualBox) or enhanced session (Hyper-V). This way you edit code on your host and run it in the VM.

TIP

You do not need the VM for most development. Only use it when testing file deletion, registry changes, or default folder handler registration. 90% of your work happens on your real machine against the test sandbox.

2.4 Configure Your Terminal

Open Windows Terminal settings and set PowerShell as your default profile. Set the starting directory to C:\Dev\nexus. This means every new terminal tab drops you straight into your project.

2.5 Environment Variables

Create a .env file in your project root (C:\Dev\nexus\.env):

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
NEXUS_SANDBOX_PATH=C:\NexusTestSandbox
NEXUS_DB_PATH=C:\Dev\nexus\nexus.db
OLLAMA_HOST=http://localhost:11434
```

The app reads these on startup. You switch NEXUS_SANDBOX_PATH to your real filesystem root only when ready for production use.

3. Understanding Claude Code

3.1 What Claude Code Actually Is

Claude Code is a command-line AI agent. You run it in your terminal, it reads your project files, writes code, creates files, runs commands, and iterates based on errors. It is not a chat window. It is an agent that operates directly on your filesystem.

When you give it a prompt, it can read any file in your project directory, create new files, edit existing files, run terminal commands (with your permission), and fix its own errors by reading compiler/runtime output.

3.2 How to Start a Session

Step 1

Open your terminal.

Step 2

Navigate to your project root:

```
cd C:\Dev\nexus
```

Step 3

Launch Claude Code:

```
c laude
```

Step 4

You are now in an interactive session. Type your prompt and press Enter. Claude Code reads your project, thinks, and starts writing code.

3.3 Key Concepts

Context window

Claude Code can see your project files, but it has a limited context window (how much text it can hold in memory at once). For large projects, it cannot see every file simultaneously. It reads files on demand. If it seems to have forgotten something, it is because that file is no longer in its active context. Re-reference the file or ask it to re-read it.

Permissions

Claude Code asks for permission before running terminal commands (like `npm install` or `python scripts`). You can approve individually or type 'yes' to auto-approve for the session. Be cautious with auto-approve during early development. Read what it wants to run.

Session continuity

Each Claude Code session is independent. When you close the terminal and start a new session, Claude Code does not remember the previous session. It starts fresh by reading your project files. This is fine. Your project files ARE the memory. As long as the code is saved, nothing is lost.

The CLAUDE.md file

You can create a file called CLAUDE.md in your project root. Claude Code reads this automatically at the start of every session. Use it to store project context, architecture decisions, coding conventions, and anything you want Claude Code to always know. This is your most powerful tool for maintaining consistency across sessions.

```
# CLAUDE.md example content:

# Nexus File Explorer

## Architecture
- Python backend (FastAPI for local web server)
- Electron frontend
- SQLite database at NEXUS_DB_PATH
- Ollama for local LLM, Claude API for cloud fallback

## Conventions
- All filesystem paths come from config.py, never hardcoded
- All file operations are logged to the operations table before executing
- Never delete files without user approval
- Test against C:\NexusTestSandbox, never real filesystem

## Current state
- Phase 1 complete: indexer and database
- Working on: Phase 2, tagging engine
- Known bugs: none currently
```

TIP

Update CLAUDE.md at the end of every work session. Add what you built, what is broken, and what to work on next. Future sessions start by reading this, so it is like leaving a note for yourself and for Claude.

3.4 How Claude Code Handles Errors

When Claude Code writes code and runs it, if it errors, it reads the error output and attempts to fix it automatically. This loop continues until the code works or it gets stuck. If it gets stuck, it asks you for help. Your job in this loop: read the error, determine if Claude Code is going in circles, and redirect if needed. Sometimes the right move is to tell it to scrap an approach and try a different one.

3.5 Slash Commands

Inside a Claude Code session, these commands are available:

- `/clear` : Clears the conversation context. Use when Claude seems confused or the context is bloated.
- `/compact` : Summarizes the conversation to free up context space while retaining key info.

- `/cost` : Shows how many tokens you have used this session and the estimated cost.
- `/help` : Lists all available commands.

4. Project Scaffolding

This is the very first thing you build. The scaffolding defines the folder structure and base files that every other feature plugs into.

4.1 Initialize the Project

Step 1

Navigate to your project folder and initialize Git:

```
cd C:\Dev\nexus
git init
```

Step 2

Create the base folder structure manually (or have Claude Code do it in your first session):

```
nexus/
backend/
__init__.py
config.py # All paths, env vars, constants
database.py # SQLite schema and queries
indexer.py # Filesystem walker
hasher.py # xxHash file hashing
tagger.py # Rules engine + AI tagging
classifier.py # LLM classification logic
watcher.py # Filesystem event watcher
mover.py # File move/delete operations
operations_log.py # Undo/history system
api.py # FastAPI local server
frontend/
package.json
electron.js # Electron main process
index.html
src/
app.js
styles.css
tests/
test_indexer.py
test_tagger.py
test_classifier.py
test_mover.py
CLAUDE.md
.env
.gitignore
requirements.txt
README.md
```

Step 3

Create the .gitignore file immediately:

```
# .gitignore
.env
```

```
*.db
*.db-wal
*.db-shm
node_modules/
__pycache__/
*.pyc
dist/
build/
.vscode/
*.log
```

Step 4

Create requirements.txt:

```
fastapi
uvicorn
python-dotenv
xxhash
watchdog
aiohttp
aiosqlite
pillow
mutagen
python-magic-bin
```

Step 5

Install Python dependencies:

```
pip install -r requirements.txt
```

Step 6

Initialize the Electron frontend:

```
cd frontend
npm init -y
npm install electron --save-dev
cd ..
```

Step 7

Make your first Git commit:

```
git add .
git commit -m "initial project scaffolding"
```

4.2 Your First Claude Code Prompt

With the scaffolding in place, start a Claude Code session and give it your first real task:

```
claude

> Read the CLAUDE.md and the project structure. Then build out
config.py with all the environment variable loading, path
definitions, and default settings. Also build the SQLite
```

```
schema in database.py with tables for: files (path, hash,
size, extension, created, modified, category, confidence,
status, original_path), tags (id, name, color, group, type),
file_tags (file_id, tag_id), operations_log (id, operation
type, source_path, dest_path, timestamp, batch_id, undone).
Use aiosqlite for async access and WAL mode.
```

Claude Code will create both files, run them to verify they work, and fix any errors. Review the output before moving on.

5. The Safe Development Environment

5.1 The Golden Rule

Never point the app at your real filesystem until the feature you are testing has been validated against dummy data at least three times. This is the single most important rule of the entire project.

5.2 Three Tiers of Testing

Tier 1: Unit tests (automated, safe)

These test individual functions in isolation. Does the hasher produce the right hash? Does the tagger assign the right tag for a .png file? These run against in-memory data or tiny temp files. No real filesystem interaction. Claude Code can write and run these freely.

Tier 2: Integration tests (sandbox)

These test features working together against your C:\NexusTestSandbox folder. The indexer walks the sandbox, the tagger tags those files, the classifier proposes moves. Files may actually get moved within the sandbox. Always restore from backup after a test round.

Tier 3: Destructive tests (VM only)

Registry modifications, default folder handler registration, file deletions, Windows service registration. These only run inside the VM. Revert to snapshot after each test.

5.3 How config.py Enforces Safety

config.py should have a SAFETY_MODE flag. When SAFETY_MODE is True (the default during development):

- All filesystem paths are prefixed with the sandbox path.
- Any attempt to write outside the sandbox raises an exception.
- Delete operations are logged but not executed (dry-run mode).
- Registry operations are skipped with a log message.

You flip SAFETY_MODE to False only when you are ready to run on real data. Claude Code should build this guard into config.py from the start.

5.4 Backup Discipline

- Before every testing session: copy the sandbox to the backup folder.
- Before every VM test: revert to the clean snapshot.
- Before every risky Git change: commit what you have so you can revert.
- Weekly: copy your entire C:\Dev\nexus folder to an external drive or cloud storage.

6. The Build Pipeline (Phase by Phase)

Build in this order. Each phase produces a working, testable piece. Do not skip ahead.

Phase 1: The Indexer and Database

Goal: walk the sandbox filesystem and populate SQLite with every file's metadata.

Claude Code prompts for this phase:

```
> Build the file indexer in indexer.py. It should async-walk the directory at NEXUS_SANDBOX_PATH, skip hidden/system files, and for each file: compute the xxhash64, extract size, extension, created date, modified date, and insert a row into the files table. Show a progress counter. Handle permission errors gracefully (skip and log).
```

Test: run the indexer against the sandbox. Open the database in DB Browser and verify the data looks correct.

Git commit when it works.

Phase 2: The Tagging and Rules Engine

Goal: assign system tags to every indexed file based on deterministic rules.

```
> Build the tagging rules engine in tagger.py. It reads all files from the database and assigns system tags based on: extension mapping (e.g. .png/.jpg/.gif -> type:image), size classification, known folder patterns (e.g. anything in AppData -> system:appdata). Store tags in the tags and file_tags tables. Include a JSON config file for the extension-to-category mapping so I can customize it.
```

Test: run the tagger, query the database for tagged files, verify categories make sense.

Phase 3: AI Classification (Local LLM)

Goal: send ambiguous files to Ollama for semantic classification.

```
> Build the classifier in classifier.py. It queries the database for files with no tags or low-confidence tags, batches them into groups of 10, and sends each batch to Ollama (llama3.1:8b at OLLAMA_HOST) with a system prompt that includes the user's folder structure and categories. Parse the response, assign tags and a confidence score. Fall back to "needs_approval" if confidence is below 0.6. Add Claude API fallback for confidence below 0.3 (reads ANTHROPIC_API_KEY from env).
```

Test: add some ambiguously named files to the sandbox, run the classifier, verify the AI's guesses.

Phase 4: The FastAPI Backend

Goal: expose all backend functionality as a local HTTP API that the frontend can call.

```
> Build the FastAPI server in api.py. Endpoints needed:
GET /files - list files with pagination, filtering, sorting
GET /files/{id} - single file detail
GET /tags - list all tags
POST /files/{id}/tags - add tags to a file
GET /search?q= - search files by name, tag, type
GET /duplicates - list duplicate groups
POST /scan/start - kick off the indexer
GET /scan/status - progress of current scan
POST /organize/propose - generate reorganization plan
POST /organize/execute - execute approved plan
POST /undo/{operation_id} - undo a specific operation
GET /history - list operation history
Run on localhost:9876.
```

Phase 5: The Electron Frontend Shell

Goal: basic Electron window with a sidebar, content area, and address bar. No file browsing yet, just the layout.

```
> Set up the Electron app in frontend/. Create the main
process (electron.js) that opens a window and loads
index.html. Build the base layout: fixed sidebar (220px,
collapsible), top bar with address/breadcrumb bar and
search input, main content area, and a collapsible right
preview panel. Use CSS variables for dark/light mode.
Default to dark mode. Keep it minimal: dark backgrounds,
single accent color (muted blue), Inter font, thin borders.
No frameworks yet, just clean HTML/CSS/JS.
```

Phase 6: File Browsing in the UI

Goal: browse files from the database, click into folders, preview files.

```
> Connect the frontend to the FastAPI backend at
localhost:9876. Implement: sidebar shows pinned folders
and tag categories. Clicking a folder fetches its contents
from /files and renders a list view (filename, size, date,
tags). Clicking a file shows its preview in the right
panel. Implement the breadcrumb bar for navigation.
Implement list view and grid view toggles. Implement
multi-select with checkboxes. Implement right-click
context menu with: open, rename, delete, tag, copy path.
```

Phase 7: The Everything Folder Watcher

Goal: background service that watches a folder and classifies new files.

```
> Build the watcher in watcher.py using the watchdog library. It watches NEXUS_SANDBOX_PATH/Downloads for new files. When a file appears and is fully written (not a partial download), it runs through the tagging and classification pipeline. If auto-sort is enabled and confidence is high, move the file to its destination (log the operation first). Otherwise, add to the needs_approval queue. Expose the queue via the API. Add a system tray icon with a popup showing recent files.
```

Phase 8: The Scan and Reorganization Pipeline

Goal: the full initial scan with cleanup suggestions and tree proposal.

```
> Build the scan pipeline that ties indexer, tagger, classifier, and deduplication together. The scan runs as a background task with progress reported via the API. After classification, generate a reorganization proposal: a JSON tree of proposed folders with file assignments. Expose via POST /organize/propose. Build the tree editor UI: two-pane view (current vs proposed), drag-and-drop to rearrange folders and files, inline rename, color-coded nodes (green=new, yellow=moved, red=needs-review). "Execute" button calls POST /organize/execute.
```

Phase 9: Search, Smart Folders, and Polish

Goal: full search with filters, smart folders, natural language search, dark/light mode toggle, keyboard shortcuts, and general UI polish.

Phase 10: Undo System and Safety

Goal: full operation history, per-operation undo, batch undo, time machine view. This should be built before you ever test against real files.

7. How to Prompt Claude Code

7.1 The Anatomy of a Good Prompt

A good Claude Code prompt has four parts:

- **Context:** what already exists, what file(s) to read or modify.
- **Task:** exactly what to build or change.
- **Constraints:** what to avoid, what patterns to follow, what libraries to use.
- **Verification:** how to test that it worked.

7.2 Example Prompts (Good vs. Bad)

Bad prompt:

```
> Make the file browser better
```

Vague. Claude Code has no idea what 'better' means. It will guess, probably wrong.

Good prompt:

```
> In frontend/src/file-browser.js, the list view currently shows filename and size. Add columns for: date modified (formatted as "Mar 15, 2024"), file type tag (pulled from the /files API response), and a small colored dot showing the AI confidence level (green > 0.8, yellow 0.5-0.8, red < 0.5). Make the columns sortable by clicking the header. Test by running the frontend and verifying the columns render with data from the sandbox.
```

Specific file, specific changes, specific format, specific test.

7.3 Multi-Step Tasks

For complex features, break them into sequential prompts within the same session. Let Claude Code finish one step, verify it works, then give the next step. Do not dump an entire feature specification in one prompt. Claude Code handles focused, incremental tasks much better.

7.4 When Claude Code Is Stuck

- If it is looping on the same error: tell it to try a different approach. Be specific about what to change.
- If the context is bloated: use `/compact` or start a new session.
- If it wrote something wrong: do not just say 'that is wrong.' Explain what the correct behavior is and paste the relevant error or wrong output.
- If it forgot project conventions: tell it to re-read `CLAUDE.md`.

7.5 Prompts to Avoid

- "Build the entire app." Too large. It will produce something shallow and broken.
- "Make it look professional." Subjective. Give specific design references or CSS properties.
- "Fix all the bugs." Which bugs? Paste error messages or describe broken behavior.
- "Do whatever you think is best." You are the architect. Claude Code is the builder. Give direction.

8. Testing Your Application

8.1 Running the Backend

```
cd C:\Dev\nexus
python -m backend.api
```

This starts the FastAPI server on localhost:9876. Leave this terminal open.

8.2 Running the Frontend

```
cd C:\Dev\nexus\frontend
npm start
```

This launches the Electron window. It connects to the backend automatically.

8.3 Running the Indexer Manually

```
cd C:\Dev\nexus
python -m backend.indexer
```

Indexes the sandbox and populates the database. Watch the progress output.

8.4 Running Tests

```
cd C:\Dev\nexus
python -m pytest tests/ -v
```

Runs all unit tests. The -v flag shows verbose output per test.

8.5 Inspecting the Database

Open nexus.db in DB Browser for SQLite or the VS Code SQLite Viewer extension. Run queries to verify data:

```
SELECT * FROM files WHERE category IS NULL;
SELECT * FROM files WHERE confidence < 0.5;
SELECT tag_name, COUNT(*) FROM file_tags GROUP BY tag_name;
```

8.6 Testing the Watcher

Step 1

Start the backend and watcher service.

Step 2

Drop a file into C:\NexusTestSandbox\Downloads.

Step 3

Watch the terminal output. The watcher should detect the file, classify it, and either auto-sort it or add it to the approval queue.

Step 4

Check the database. The file should have a row with tags and a proposed destination.

8.7 Testing the Reorganization

Step 1

Run the full scan pipeline against the sandbox.

Step 2

Open the tree editor in the UI.

Step 3

Review the proposal. Make adjustments.

Step 4

Execute the reorganization.

Step 5

Open C:\NexusTestSandbox in Windows Explorer and verify files moved correctly.

Step 6

Test undo: revert the reorganization from the history panel. Verify files return to original locations.

9. Making Edits Efficiently

9.1 Small Edits: Do Them Yourself

If you need to change a color, fix a typo, adjust a padding value, or rename a variable, do it in VS Code directly. Firing up Claude Code for a one-line change is slow. Learn to navigate your own codebase for small fixes.

9.2 Medium Edits: Targeted Claude Code Prompts

For changes like adding a new column to the database, modifying an API endpoint, or restyling a component, start a Claude Code session and give a focused prompt that references the specific file(s) to change.

```
> In backend/database.py, add a new column "last_accessed"
(datetime, nullable) to the files table. Update the
indexer to populate this field using os.path.getatime().
Update the /files API endpoint to include this field in
the response. Update the frontend list view to show it
as a sortable column.
```

One prompt, touches four files, but a single coherent change.

9.3 Large Edits: Plan First

For big changes (new features, major refactors), plan BEFORE prompting. Write out what needs to change and in which files. Then feed that plan to Claude Code as a multi-step series. Commit between each step so you can revert.

9.4 The Edit-Test-Commit Loop

Every change follows this cycle:

Step 1

Make the edit (manually or via Claude Code).

Step 2

Test: run the relevant tests, visually check the UI, or run the specific feature.

Step 3

If it works: git add and commit with a descriptive message.

Step 4

If it is broken: fix or revert (git checkout -- filename to discard changes to a specific file, or git reset HEAD~1 to undo the last commit).

Never let more than an hour of work go uncommitted.

10. Version Control with Git

10.1 Essential Git Commands

You need roughly 10 Git commands for this project:

Command	What It Does
<code>git status</code>	Shows which files are changed, staged, or untracked.
<code>git add .</code>	Stages all changed files for commit.
<code>git add filename</code>	Stages a specific file.
<code>git commit -m "message"</code>	Creates a snapshot of all staged changes.
<code>git log --oneline</code>	Shows commit history, one line per commit.
<code>git diff</code>	Shows what changed in unstaged files.
<code>git checkout -- filename</code>	Discards changes to a specific file (reverts to last commit).
<code>git reset HEAD~1</code>	Undoes the last commit but keeps the changes as unstaged.
<code>git branch feature-name</code>	Creates a new branch for experimental work.
<code>git checkout branch-name</code>	Switches to a different branch.
<code>git merge branch-name</code>	Merges a branch into your current branch.

10.2 Branching Strategy

Keep it simple. You are a solo developer.

- **main**: always working. Never commit broken code directly to main.
- **feature branches**: one per major feature (e.g., feature/tree-editor, feature/watcher). Work here, merge to main when done.
- If a feature branch gets messy, abandon it and start a new one from main.

10.3 Commit Message Convention

Be descriptive but brief:

```
git commit -m "add file indexer with xxhash and progress counter"
git commit -m "fix: watcher ignoring .part files during download"
git commit -m "ui: add dark/light mode toggle to settings"
```

10.4 Remote Backup (GitHub)

Step 1

Create a private repository on GitHub.

Step 2

Link it:

```
git remote add origin https://github.com/yourname/nexus.git  
git push -u origin main
```

Step 3

After every work session, push:

```
git push
```

This gives you a cloud backup of every version of your code.

11. Sharing the Project

11.1 Sharing with Friends (Developer)

If your friends are comfortable with a terminal:

Step 1

They clone the repo:

```
git clone https://github.com/yourname/nexus.git
cd nexus
```

Step 2

They create their own .env file with their own API key and paths.

Step 3

They install dependencies:

```
pip install -r requirements.txt
cd frontend && npm install && cd ..
```

Step 4

They run the app:

```
python -m backend.api
# in a second terminal:
cd frontend && npm start
```

11.2 Sharing with Friends (Non-Developer)

Package the app so they just double-click an installer:

Step 1

Use PyInstaller to bundle the Python backend into a standalone .exe:

```
pip install pyinstaller
pyinstaller --onedir backend/api.py --name nexus-backend
```

Step 2

Use electron-builder to package the frontend:

```
cd frontend
npm install electron-builder --save-dev
npx electron-builder --win
```

Step 3

This produces a dist/ folder with an installer (.exe). The installer includes the frontend. You also need to bundle the backend .exe alongside it.

Step 4

Write a startup script that launches the backend .exe in the background and then opens the Electron app.

TIP

Packaging is a Phase 11+ concern. Do not worry about it until the app is feature-complete and stable on your own machine.

11.3 Sharing the CLAUDE.md for Collaboration

If a friend wants to develop alongside you with Claude Code, they need the CLAUDE.md file in the repo. Keep it updated so their Claude Code sessions have the same context as yours. Divide work by feature branch so you do not conflict.

12. Updating and Maintaining the App

12.1 Adding New Features

Step 1

Create a feature branch:

```
git checkout -b feature/new-thing
```

Step 2

Update CLAUDE.md with the planned feature details.

Step 3

Work with Claude Code to build it (using the prompting patterns from Chapter 7).

Step 4

Test thoroughly against the sandbox.

Step 5

Merge to main:

```
git checkout main
git merge feature/new-thing
git push
```

12.2 Updating Dependencies

Periodically update your Python and Node packages:

```
pip install --upgrade -r requirements.txt
cd frontend && npm update && cd ..
```

After updating, run your tests to make sure nothing broke. If something did, revert.

12.3 Updating the LLM Models

When Ollama releases newer models:

```
ollama pull llama3.2:8b
# or whatever the latest version is
```

Update the model name in config.py. Test the classifier against your sandbox to verify the new model performs at least as well as the old one before switching permanently.

12.4 Database Migrations

When you add a column or table to the SQLite schema, existing databases on your machine (and your friends' machines) need updating. Build a migrations system early:

- Store a schema_version in a metadata table.

- On app startup, check `schema_version` against the expected version in code.
- If mismatched, run `ALTER TABLE` statements to bring the schema up to date.
- Claude Code can build this migration system for you. Ask it early in Phase 1.

12.5 Pushing Updates to Friends

If your friends run from the repo (developer setup):

```
# They run:
git pull
pip install -r requirements.txt
cd frontend && npm install && cd ..
```

If they use the packaged installer, you need to rebuild the package and send them a new installer. Consider a simple auto-update mechanism later: the app checks a URL for the latest version number and prompts to download if outdated.

13. Deploying to Other Machines

13.1 First Install on a New Machine

Step 1

Install Python, Node.js, Git (same as Chapter 1).

Step 2

Install Ollama and pull the model.

Step 3

Clone the repo or copy the project folder.

Step 4

Create the .env file with machine-specific paths and API key.

Step 5

Install dependencies (pip install, npm install).

Step 6

Run the initial scan to build the database for this machine's filesystem.

Step 7

Set SAFETY_MODE to False in .env only after verifying the scan results look correct.

13.2 Cross-Machine Config Differences

Things that change per machine (all in .env, never in code):

- NEXUS_SANDBOX_PATH (or the real filesystem root)
- NEXUS_DB_PATH
- ANTHROPIC_API_KEY
- OLLAMA_HOST (usually the same, but could differ on a network setup)
- Custom file type priorities (these could be a separate JSON config file that IS committed to Git, with per-user overrides in .env)

13.3 The Installer (Future)

For a truly polished deploy, build an NSIS or WiX installer that bundles everything: the Python backend (frozen with PyInstaller), the Electron app (packaged with electron-builder), the Ollama runtime, and the default model. The installer runs the initial setup wizard (Chapter 10 of the feature list). This is a v2 or v3 goal.

14. Troubleshooting Common Issues

Claude Code says 'context window full'

Your project has gotten large enough that Claude Code cannot hold it all at once. Use `/compact` to summarize the conversation. For very large projects, give Claude Code specific file paths instead of letting it scan the whole project. Keep `CLAUDE.md` concise.

Python module not found errors

Make sure you are running from the project root (`C:\Dev\nexus`) and using the correct Python environment. Try: `python -m backend.api` instead of `python backend/api.py`. The `-m` flag handles module resolution correctly.

Electron window is blank

The backend probably is not running. Start it first (`python -m backend.api`), then launch Electron. Check the Electron dev tools (`Ctrl+Shift+I`) for network errors.

Ollama is slow or unresponsive

Check that the Ollama service is running (`ollama list` should show your model). If classification is too slow, reduce batch sizes in `classifier.py`. Check GPU utilization. If Ollama is using CPU instead of GPU, reinstall with GPU support.

Database is locked

SQLite does not handle concurrent writes well. Make sure only one instance of the backend is running. If the database got corrupted from a crash, check for `.db-wal` and `.db-shm` files. Delete them (the data they contain will be lost) and the database reverts to its last consistent state.

Files were moved to the wrong place

Use the undo system. Open the history panel, find the operation, and click undo. If the undo system is not built yet, check the `operations_log` table in the database. It records source and destination for every move.

Git merge conflicts

If you and a friend edit the same file, Git will flag a conflict. Open the file in VS Code. It highlights the conflicting sections. Pick the version you want, save, then `git add` and `git commit`.

Windows blocks the Electron app

SmartScreen may flag an unsigned app. Click 'More info' then 'Run anyway.' For distribution, you would need a code signing certificate (expensive, not needed for personal use).

The watcher misses files

Some apps write files in a way that does not trigger filesystem events. Add a periodic polling fallback in `watcher.py` that checks the folder every 30 seconds for files not yet in the database.

API key exposed in Git

If you accidentally committed your `.env` file: rotate the key immediately at console.anthropic.com. Remove it from Git history: `git filter-branch` or use BFG Repo-Cleaner. Then re-push.

15. Reference: File Structure and Tech Stack

15.1 Complete Tech Stack

Layer	Technology	Purpose
Frontend	Electron + HTML/CSS/JS	Desktop window, UI rendering
Backend	Python + FastAPI	Business logic, API server
Database	SQLite (aiosqlite)	File index, tags, history
Local AI	Ollama (Llama 3.1 8B)	File classification, semantic tagging
Cloud AI	Claude API (Anthropic)	Fallback for hard classifications
File Hashing	xxHash (xxhash lib)	Deduplication, integrity checks
File Watching	watchdog library	Everything Folder monitoring
Metadata	Pillow, mutagen, python-magic	Image EXIF, audio tags, MIME types
Packaging	PyInstaller + electron-builder	Distribution (future)
Version Control	Git + GitHub	Code history, collaboration

15.2 Database Schema (Quick Reference)

```
files
id INTEGER PRIMARY KEY
path TEXT UNIQUE NOT NULL
filename TEXT NOT NULL
extension TEXT
size INTEGER
hash TEXT
created DATETIME
modified DATETIME
category TEXT
confidence REAL DEFAULT 0.0
status TEXT DEFAULT 'unprocessed'
original_path TEXT

tags
id INTEGER PRIMARY KEY
name TEXT UNIQUE NOT NULL
color TEXT
tag_group TEXT
tag_type TEXT -- 'system', 'ai', 'user'

file_tags
file_id INTEGER REFERENCES files(id)
tag_id INTEGER REFERENCES tags(id)
PRIMARY KEY (file_id, tag_id)
```

```
operations_log
id INTEGER PRIMARY KEY
op_type TEXT -- 'move', 'delete', 'rename', 'tag'
source_path TEXT
dest_path TEXT
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
batch_id TEXT
undone BOOLEAN DEFAULT 0
```

16. Reference: Keyboard Shortcuts and Commands

16.1 Development Commands (Terminal)

Command	What It Does
<code>claude</code>	Start a Claude Code session in the current directory
<code>claude 'prompt here'</code>	Run a one-shot Claude Code command
<code>python -m backend.api</code>	Start the FastAPI backend server
<code>cd frontend && npm start</code>	Launch the Electron frontend
<code>python -m backend.indexer</code>	Run the file indexer manually
<code>python -m pytest tests/ -v</code>	Run all unit tests
<code>ollama run llama3.1:8b</code>	Test Ollama interactively
<code>git status</code>	Check for uncommitted changes
<code>git log --oneline -10</code>	View last 10 commits

16.2 Claude Code Session Commands

Command	What It Does
<code>/clear</code>	Wipe conversation context, start fresh in the same session
<code>/compact</code>	Summarize conversation to free context space
<code>/cost</code>	Show token usage and estimated cost for this session
<code>/help</code>	List all available slash commands
<code>Ctrl+C</code>	Cancel the current Claude Code operation
<code>exit</code> or <code>Ctrl+D</code>	End the Claude Code session

16.3 Application Keyboard Shortcuts (Default)

Shortcut	Action
<code>Win+Shift+E</code>	Summon/hide the Nexus window (global, configurable)
<code>Win+Shift+D</code>	Open latest download (global, configurable)
<code>Ctrl+T</code>	New tab
<code>Ctrl+W</code>	Close tab

Ctrl+F	Focus search bar
Ctrl+K	Open command bar (natural language)
Ctrl+P	Toggle preview panel
Ctrl+B	Toggle sidebar
Alt+Left	Navigate back
Alt+Right	Navigate forward
Alt+Up	Go up one folder
Ctrl+A	Select all files in current view
F2	Rename selected file
Delete	Delete selected file(s)
Ctrl+Z	Undo last app operation
Ctrl+1/2/3/4	Switch view mode (list/grid/column/detail)
Ctrl+Shift+D	Toggle dark/light mode
Ctrl+,	Open settings

End of Manual